

- Later, after testing, you can expand to allow multiple simultaneous active parents (“stacked Sub-Orders”) in PortalX Advanced or EmpowerBot.
-

🌟 4. Step-Up / Step-Down Logic

Yes 🦞 — that belongs in **PortalX PRO or EmpowerBot**, not LITE.

It’s inherently **reactive automation** (monitoring market tickers, calculating deltas, and dynamically updating order prices).

PortalX LITE is meant for **guided manual intelligence**, so it’s better to leave that feature dormant until the automation layer can be handled by a background script or divine-tech parallel logic.

🌟 5. Where You Are Now (and Why It’s Great)

You’ve got the foundation of an **intuitive, human-trackable order life cycle** that perfectly matches the Stellar order model:

```
Parent Order (offer_id)
├── Sub-Order #1 (Edit)
├── Sub-Order #2 (Edit)
├── ...
└── Effective Price → feeds Pending To Do
    ↓
    Next Order (new parent, new offer_id)
```

Once you’ve logged a few real examples through Lobstr, you’ll see the natural flow. From there, we can refine your formulas for how Effective Price and Available Assets carry forward.

Would you like me to write a short **example scenario** (with made-up numbers) showing a parent order with two Sub-Orders and how the Effective Price and Available Assets would flow into the Next Order? It might help visualize how it all ties together in your doc.

You said:

I’ve already got sample scenarios in my sandbox Order Log spreadsheet Poof 🧙🏻💡🦞 We just need to figure out how the offer_id is used to extract the right data from the network. I actually think that PortalX LITE parents can be maintained even if new offer_ids get thrown in the mix, but my brain is just not able to process it right now. I believe once we write the basic Next Order logic it will snap into my brain how to handle it. You see, the Order IDs and Sub-Order IDs are what will make it possible, because they describe exactly what an order’s price is. It’s just a matter of pulling the right amounts that fill and remain from the Stellar network. If a Sub-Order gets a new offer_id, it doesn’t matter, because the spreadsheet knows which offer_id to look at, because it gets assigned to the same row of the Sub-Order. Anyway, I think I know how I want to handle it now, so this discussion helped. The tricky thing is matching offer_ids to Order IDs and Sub-Order IDs, because the logic will have to search for the offer_id based on data that PortalX LITE gives the user to create the order like price and

fixed asset amounts/totals, because I don't want the user to have to go look for the `offer_id` on StellarExpert. The other day I had thought of a way to do it, but I guess I didn't write it down in my notes, so that's a bummer. I'm going to call it a night now. I'll let you know if I need any more help.

ChatGPT said:

That sounds perfect, Boo Boo 🍌🐼 — you've actually got the right mental map forming. Once you sketch the base logic, the network-lookup piece will start feeling obvious again, because Stellar's effects and offers endpoints make it possible to triangulate the right `offer_id` from the attributes you already track (asset pair + price + amount + timestamp).

When your brain is fresher, you can play with a flow like:

1. PortalX LITE logs the order parameters the moment the user creates it.
2. Later, the script queries recent offers for that wallet and asset pair, filters by matching price/amount range, and returns the corresponding `offer_id`.
3. That `offer_id` then links every future Sub-Order fill and effect event automatically.

You're right that once the "Next Order" logic is cleanly written, this mapping piece will click back into place — it's just pattern-recognition fatigue right now.

Rest your brain; you've done a lot of heavy conceptual lifting. We can tighten that extraction routine together once you're rested and ready. Sleep well, and let the Multiverse do some quiet compiling for you tonight. 🌙🐼

You said:

Good morning Poof 🍌🐼 So last night after I talked to you I didn't end up going to sleep right away. I was hungry, so I made something to eat and watched four episodes of a fairly new Netflix series called "Nobody Wants This" or something like that. It's about a woman who does a podcast with her sister that is mostly focused on sex, but that the main character believes is really about empowerment. She's basically not religious or spiritual, but ends up falling for a young rabbi who doesn't act like people would think a rabbi would act, but of course his family doesn't approve of the new relationship, especially since he has recently dumped his Jewish princess girlfriend that everyone thought he was going to marry. So that's the show. But anyway, while watching (or after, I can't remember) I started thinking about my dilemma with the next orders, and ended up coming into my room and drawing out a scenario, and now I've kind of changed my mind about what I thought before. I think we CAN make it so that if people WANT to manage Sub-Orders individually, and have like Sub-Cycles (though we don't need to use the term), then I believe that is totally doable. Here's why I think this could work... Because of the way I have set up the Order Log spreadsheet now it should be possible to keep track of Orders and Sub-Orders even if they split into Sub-Cycles. I just have to figure out the proper logic of when to create a Sub-Order, but basically my whole system is based on making profitable Cycles, and the way we've coded the Order and Sub-Order IDs is what makes it possible to keep track of Cycles even if they are fragmented in their completion. Since I haven't built the "Log Orders" prototype section of the HTML web app yet, I can bring this new revelation into the design. It needs to work a bit like an algorithm. The "Pending To Do" sub-section is going to list all of the Orders and Sub-Orders that have either filled or partially filled, and sometimes there will be overlap because the Sub-Order(s)

may be part of a parent Order listed, so my algorithmic approach needs to know that and ask the user if they'd like to manage Next Orders on the parent Order level or the Sub-Order level which is more precision, but more complex and requires more work and XLM Reserve. In fact, we may even consider adding a setting which would then filter out the parent Orders or the Sub-Orders (when applicable) from the "Pending To Do" list to prevent confusion if they'd user doesn't want to have to make a choice each time they go to manage and log orders. But people like me might want both options so I can deal with each on an individual basis. So let me explain how this works... On our Google Sheets document we have both a "Next Order Form", and an "Update Order Form". But technically the "Next Order Form" is really a "New Order Form" like you were mentioning yesterday. Normally to make a next trade if the user was using only a portion of the assets received from the previous partially filled Order, then they'd place a partial Next Order, and then use the "Update Order Form" to add the rest of the assets to the Next Order as they become available, so that next Order is associated with only one new offer_id. And if the user wanted to change the price of that Order then if any portion of that Order had filled it would be split off as the first filled Sub-Order, adding "SO1" to the parent Order ID, and logging it in a separate row while also creating "SO2" with the new price. But these Sub-Orders are still associated with the offer_id of the parent, so the API calls would use that offer_id to get update data. But theoretically, if one wanted to treat Sub-Orders as Sub-Cycles, they could use the Next Order form logic to manage each Sub-Order which would create multiple separate Sub-Orders that have different offer_ids, but they could still be tracked the same way in our PortalX LITE logic, but instead of pulling their data from the offer_ids associated with the parent, they pull it from the new offer_id associated with their Sub-Order ID since I added a "Stellar offer_id" column to the "Log Orders" sheet. So do you think you could code this logic? Especially after I can show you the revised "Log Order" sheet? About a week or so ago I had reformatted that sheet by separating data for buy and sell orders which shows for each what the amount (quote assets) or total (base assets) of the fixed asset was at the start and remaining, while also showing how much of the other asset was received. This made it less confusing, easier to track, and easier to determine the "Effective Price" for the parent, but like I said yesterday, Sub-Orders don't have "Effective Prices", so if you're micromanaging and treating Sub-Orders with individual Cycles then you would use the Sub-Order ID for the previous order, and it would calculate the next price based on the previous Sub-Order's price, and you would possibly end up making multiple Next Orders for each Sub-Order of a parent, each with their own offer_id. BUT you could also simplify it and just submit the parent Order ID of the previous order and use the Effective Price to determine the Next Order, and then just keep using the "Update Order" logic to add the available assets from the Sub-Order to the next parent order so there is only one offer_id. I'm thinking that we could make the latter case the default, and as people learn or what to get more sophisticated, we could have tutorials that unlock these advanced features.

ChatGPT said:

Thought for 10s

This is excellent — your realization is exactly right and the design you described is feasible. You've found a clean way to support both (A) the simple default flow most users will want and (B) an advanced, micro-managed flow for power users who want to treat sub-orders as independent small cycles. Below I'll:

1. confirm the behavior we'll implement (default vs advanced)
2. list the **sheet columns** needed and how they're used (so the code can map correctly)
3. describe the **runtime algorithm** for (a) detecting parent vs sub, (b) computing Effective Price, (c) deciding next-order source (parent or sub), and (d) offer_id matching on the Stellar network
4. provide a compact set of **Apps Script / JS function outlines** (pseudo-code / implementation plan) you can test modularly
5. give the recommended **UI interactions** for the Pending → Next flows and how the forms should behave
6. next steps for you (what to share & how I'll proceed)

I tried to keep this practical and implementable — no big monolithic script, just small modular functions to test and wire up.

1) Desired behavior (summary)

Default (recommended for most users)

- PortalX LITE uses the **parent Order ID** and the parent's **EffectivePrice** (calculated from all fills / subs that actually filled so far) as the baseline for recommending the next order.
- Users create a next order parent (one offer_id) and then can **add** remaining filled assets to that next parent via Update Order (until parent depleted) — simple linear flow.

Advanced / power user mode (optional)

- PortalX LITE exposes Sub-Orders (SO1, SO2, etc.) as independently manageable items. The user can pick a Sub-Order as the source for a Next Order; the next recommended price will be based on that Sub-Order's price (not the parent EffectivePrice).
- Each managed Sub-Order can end up with its own offer_id (stacking / new offer). UI gives power user the explicit choice: "Use Parent (combined effective price)" or "Use Sub-Order (SO1)".

Default mode is the recommendation. Advanced mode can be unlocked/selected per-account.

2) Sheet columns (required / strongly recommended)

(Use names close to these for clarity — mapping is crucial)

Order Log (core)

- A: Row index (sheet)
- B: Unique ID (random)
- C: Order ID (Parent) — rename to **Order ID (Parent)** if you prefer
- D: Sub-Order ID (e.g., CSSB1-C1-OS-SO1) or blank for parent rows
- E: Stellar offer_id (the network offer id for the row) — may be blank until we resolve it
- F: Market (e.g., AFR/USDC)
- G: Quote Asset
- H: Base Asset
- I: Type (Buy / Sell)
- J: Price (this row's price) — IMPORTANT: parent shows original price, SO rows show their price
- K: Fixed Start (Start fixed asset amount: Amount for Sell or Total for Buy) — the asset that is fixed in the order at start
- L: Fixed Remaining (what remains, updated by Update Order)
- M: Variable Received Total (the amount of the other asset actually received so far from fills on this parent/sub aggregate)
- N: Effective Price (computed only on parent rows; = $\text{totalValueReceived} / \text{totalFixedInitiallyUsedWhenFilledSoFar}$)
- O: Status (Active / Partially Filled / Filled)
- P: % Filled — computed from FixedRemaining vs FixedStart (100% when FixedRemaining = 0)
- Q: DateTime Created
- R.: any metadata, notes

Notes:

- Sub-orders are separate rows whose **Order ID (Parent)** value is the same as parent; the **Sub-Order ID** identifies them. For simple parent-only flows, Sub-Order ID column will be blank.
 - Keep the mapping consistent — scripts will look up Type and Fixed columns to decide which field is the fixed one for the order.
-

3) Algorithms (high level)

3A — Effective Price computation (parent)

When any fills have actually occurred (across parent and any sub-orders), compute:

$$\text{EffectivePrice_parent} = (\sum (\text{price_i} \times \text{qty_received_i})) / (\sum \text{qty_received_i})$$

Where:

- `price_i`, `qty_received_i` — actual executed fills (we approximate these from the network or from user-entered remaining/received values).
- For this sheet, we can compute using `VariableReceivedTotal` (amount actually received so far) and `FixedFilledTotal` (`FixedStart` – `FixedRemaining`). But simpler: compute incremental each time an update is logged by capturing `sum(quantity × price)` and `sum(quantity)`.

Use case: `EffectivePrice` is used as the `lastOrderPrice` for Next Order recommended price.

3B — When to treat parent vs sub as the source

- Present user a short choice by default (Parent recommended experience): “Use Parent (Effective Price)” — single click.
- If user opens the advanced panel and clicks a specific Sub-Order entry, treat that sub as the source. That sub’s `Price` serves as previous order price for next order recommendation. (Sub-orders don’t have `EffectivePrice` — they have their own per-sub price.)

3C — Offer_id matching to network (how to find the correct offer_id)

Goal: given the logged order parameters the user created (price, fixed start, market, timestamp-ish), find matching network offer(s) for the account.

Algorithm:

1. Query account offers from Horizon (or Lobstr Horizon) for that account + market (asset pair). You can limit by paging token / time window.
2. Filter offers for price within tolerance (small floating tolerance because of path payments/fees).
3. If multiple matches, filter by fixed start amount (or by approximate base/quote totals), and use the most recent one (created timestamp) or ask user to confirm.
4. Once matched, write `offer_id` into the Order Log row (and propagate to existing Sub-Order rows that share parent).

If user edits an order in Lobstr and Stellar created a **new** offer_id (e.g., user canceled parent and created new order), then the same matching algorithm will find the new offer_id and update.

3D — Creating Next Order

Two key flows:

A) Next Order using Parent EffectivePrice:

- Compute recommended price = EffectivePrice $\times$ (1 + recommended_sell_pct) for Sell, $\times$ (1 + recommended_buy_pct) for Buy.
- Create a **Next Parent Order row** (Order ID increments Cn etc.), set FixedStart = user-entered (or auto-calculated partial), Status Active.
- User copies price & fixed amount into Lobstr and places order. Once placed, run offer_id matching and update.

B) Next Order per Sub-Order:

- Same as above but base price is the sub price (sub row Price) and FixedStart is the variable available derived from that sub when it filled.

4) Function outlines / modular Apps Script pseudo-code

(These are modular so you can test piece by piece.)

Utilities (shared)

```
// simple tolerance compare for floats
function near(a, b, tol = 1e-8) { return Math.abs(a - b) < tol; }
```

findOfferIdForOrder(params)

- Inputs: account, market, price, fixedAmount, timeWindow (optional)
- Returns: matched offer_id or null

Pseudo:

```
function findOfferIdForOrder({account, market, price, fixedAmount, timeWindow}) {
  // call Lobstr Horizon: /accounts/{account}/offers?
  &cursor=now&limit=200&order=desc
  // parse results
  // for each offer:
  //   check offer.asset_pair == market
  //   check near(offer.price, price, tolerance)
```

```

    // check near(offer.amount, fixedAmount, tolerance) // amount is offer amount
on book
    // pick best candidate (closest time, or closest amount)
    // return offer_id or null
}

```

computeEffectivePriceForParent(orderParentRow)

- Inputs: the parent row (with child sub order rows)
- Reads all fills (from sub rows or a separate fills table)
- Returns EffectivePrice and writes to parent row.

Pseudo:

```

function computeEffectivePriceForParent(parentOrderID) {
    // get all rows with Order ID (Parent) == parentOrderID and (Status != 'Deleted')
    // sumValue = 0; sumQty = 0;
    // for each row (parent + sub rows):
    //     qtyFilled = FixedStart - FixedRemaining // fixed asset used so far on that
row
    //     if qtyFilled > 0:
    //         value = qtyFilled * row.price // careful: price unit orientation must be
consistent
    //         sumValue += value
    //         sumQty += qtyFilled
    // if sumQty > 0:
    //     eff = sumValue / sumQty
    //     write eff to parent EffectivePrice column
    // return eff
}

```

createNextOrderFromSource(sourceRow, options)

- sourceRow is either parent row or sub row.
- options: nextFixedAmount (entered by user), isPartial (true/false), isBuyOrSell
- Writes a new row for next parent order with generated Order ID and recommended price.

Pseudo:

```

function createNextOrderFromSource(sourceRow, nextFixedAmount, userPref) {
    let basePrice = (sourceRow.isParent ? sourceRow.EffectivePrice :
sourceRow.Price);
    let recommended = computeRecommendedPrice(basePrice, sourceRow.Type);
    // build new Order ID: same CSSB/CSBS prefix + next C number + OS/OB etc.
    // insert row into Order Log with status Active, Unique ID, recommended price,
FixedStart = nextFixedAmount
    // return new row info for UI display (price, orderID, uniqueID)
}

```

updateOrderFromForm(formData)

- Called by “Log Updated Order” button

- If price unchanged: update FixedRemaining and VariableReceivedTotal and recalc EffectivePrice.
- If price changed: create Sub-Order rows and move filled part to SO1 (100% filled) and remaining to SO2 etc. (you already have this structure in sample sheet; reflect it exactly).

Pseudo (simplified):

```
function updateOrderFromForm(formData) {
  // find rowIndex for orderID parent/sub
  // determine Type from Order Log
  // read fixedRemainingField from form (user enters remaining fixed asset)
  // if newPrice == existingPrice:
  //   // simple update
  //   set FixedRemaining = form.fixedRemaining
  //   set VariableReceivedTotal = computed from fills (or user input)
  // else:
  //   // price changed => create new sub order rows in sheet
  //   createSubOrderRows(parentRow, filledQty, remainingQty, oldPrice, newPrice);
  // recompute parent effective price
}
```

5) UI behavior & suggested flows (Pending → Next)

Dashboard / Pending To Do:

- Show a filtered list (panels) of “Pending” items — default shows parents with filled portion > 0 or partially filled.
- Add a filter toggle: [Show Sub-Orders] ON/OFF. (Default OFF — keep it simple for most users.)
- For each entry show:
 - Parent ID or Sub ID, status (Partially Filled), FixedRemaining, VariableReceivedTotal, EffectivePrice (for parents) or Price (for Sub).
- Clicking an item → open a compact modal:
 - Show recommended action(s):
 1. “Create Next Order (Use Parent EffectivePrice)” — button
 2. If SubOrders are present and advanced mode on: list each Sub with button “Use this Sub as next order source (advanced)”.
 3. “Open Update Order Form” — for adding remaining fixed to an existing next order.
- When user clicks Create Next:

- Show numeric input for Fixed amount to use (suggest default = VariableReceivedTotal or max available).
- Show recommended price (editable).
- Buttons: Copy Price, Copy Amount, Place in Lobstr (info — opens Lobstr market link), Log Order (writes to sheet).
- After “Log Order”:
 - Fire the function that finds offer_id & writes to the new order row (or marks it pending until user confirms it was placed).

Important: every step persists to sheet so you can always re-run the matching script (idempotent).

6) Offer_id / network matching detail (practical issues)

- Because Stellar/Lobstr sometimes returns multiple ambiguous results, the script should:
 1. try exact match (price, amount, recent timestamp),
 2. if ambiguous, choose the nearest timestamp / exact amount,
 3. if still ambiguous, mark row `offer_id = UNRESOLVED` and surface a short UI with two matches and allow user to confirm.
 - Always log the `offer_id` found; this is what links later effect/transactions queries to determine fills and recompute EffectivePrice.
-

7) Suggested test order for modular development

1. Implement and test `computeEffectivePriceForParent()` on static sample data in a test sheet.
2. Implement `findOfferIdForOrder()` against Lobstr Horizon (test using a single known transaction/offer hash).
3. Implement `updateOrderFromForm()` for the **no-price-change** flow. Test manually.
4. Implement the price-change flow that creates Sub-Order rows. Test with sample data.
5. Implement the UI flows (Pending list → modal → create next) and hook to `createNextOrderFromSource()`.

6. Add advanced mode toggle and Sub-Order management.

This modular approach lets you test each piece separately — which is exactly what you wanted.

8) Example: Decision tree for the Next Order choice (simple)

- If parent has any fills (`VariableReceivedTotal > 0`):
 - If user preference = `ParentDefault` → use parent `EffectivePrice`
 - Else if user selects `SubOrder` explicitly → use that sub price
 - If parent has no fills (brand new) → use parent price as usual.
-

9) What I can do next for you (when you're ready)

- If you share the revised **Order Log** sheet (or a copy) with the exact column headers you want to use, I will:
 1. write the Apps Script functions above mapped exactly to your column indices/names, and
 2. produce two lightweight test scripts you can run:
 - `test_computeEffectivePrice()` — demonstrates effective price computation on your sample data and prints output.
 - `test_findOfferId()` — sample code for calling Lobstr Horizon and matching an offer (I'll provide the call, you can run in your container and confirm access).
 - Or, if you prefer, I can provide the complete code snippets for the functions above adapted to the sheet names/columns you already told me about (I can map them now if you confirm the final header names). But given how many small header-name variants you've iterated through, sending me the final Order Log copy will make the implementation faster and less error-prone.
-

10) Quick checklist for your sheet before sharing (so I can code right away)

- Confirm exact column headers (copy/paste the top row into the chat or share the file):
 - Order ID (Parent), SubOrder ID, Stellar offer_id, Type, Price, FixedStart, FixedRemaining, VariableReceivedTotal, EffectivePrice, Status, %Filled
- Make sure sub-orders are separate rows and have the same Order ID (Parent) text as their parent.
- Add a small test dataset with:
 - one parent with no fills,
 - one parent with partial fills and one sub-order,
 - one parent with multiple sub-orders (simulating price changes).
This will let me test computeEffectivePrice and the update logic.

If you want, I'll also draft the small modal UI text and buttons exactly as they should appear (labels for TalkBack accessibility included) — so the Pending modal is friendly for visually impaired users (I'll ensure copy buttons have readable labels and accessible names).

You're absolutely right: default parent EffectivePrice flow first, advanced sub-order micro-management as a deliberate opt-in. That keeps PortalX LITE simple for most people but powerful for those who want it — a perfect balance.

Would you like me to prepare the two test scripts (`computeEffectivePriceForParent`, `findOfferIdForOrder`) mapped to the column names above so you can paste them into your test project and run? If so, paste your final header row here (or share the sheet) and I'll adapt the code to the exact indices.

You said:

Whew! 😊 That was a lot Poof 💜🌟🐵 but I think I understood it all, and basically am confident this is doable. Here's what I'm thinking about the UI. I need to add a button to each "Pending To Do" item that says something like "Manage Order Now" which will automatically take them to the Log Orders section and will have already run that Order ID or Sub-Order ID and figured out how much of the fixed asset is available and what the default recommended price is. If it was a parent Order that was the previous order, and this is the first "Next Order", then it will use the "Next Order" algorithm and produce a new next order with a new offer_id. If it is not the first Next Order then the user is given the option to add the assets to the previously created next parent (recommended) or create a new next order at a new price which will trigger the Sub-Order algorithm and create a next Sub-Order, and turn the first order into a Sub-Order, but not a filled one. I want to share my "Sandbox Sample Order Log" sheet with you in its Google Sheets form, but it seems you're not able to process that. When I turn it into

a .xlsx document it seems to lose features. Here's the thing... When a new Sub-Order is created, I don't want it created just in the next available row. I want it to be placed in order, grouped with the other Sub-Orders just below the parent, so the logic has to be dynamic and be able to adapt to rows being changed and must locate Orders and Sub-Orders by their IDs and then figure out what row to pull data from. Like I came up with an easy way to calculate the Effective Price just by using data in certain columns, so rather than using a complex formula to calculate "Effective Price" you use formulas to calculate amounts remaining and amounts received and use those to calculate Effective Price. Like I already came up with this before. So you're right. Before you code anything, let me figure out how to give you what I have. That's essentially what the "Ultimate" document was supposed to be. I'm thinking maybe before I have you start anything, let me work with what you've given me so far. Now that I know what you will need to code this.

ChatGPT said:

Thought for 6s

Perfect — this all makes total sense and you're thinking about all the right details. Love the "Manage Order Now" idea — it's exactly the sort of affordance that will make PortalX LITE feel effortless for most users while still letting power users drill down.

Below is a compact, practical checklist and a couple of small, concrete implementation notes (including a tiny Apps Script snippet) you can keep for when you're ready to hand me the sheet or code. No heavy lifting from me yet — just tools and clarity so your next pass is smoother.

Quick confirmation of the flow you described

- Each Pending panel has a **Manage Order Now** button.
- Clicking it opens the **Log Orders** screen and:
 - pre-loads the selected Parent or Sub-Order ID,
 - computes how much fixed asset is available,
 - proposes a default recommended price (Parent → EffectivePrice; Sub → Sub price),
 - offers the two main choices (default: create a new Next Parent; advanced: add to existing Next Parent or create a new Next Sub-Order).
- If the user chooses to create a Sub-Order, the sheet code will **insert the new SO row grouped immediately under its parent** and keep the parent + its SOs visually and logically together.

Small UX / accessibility suggestions (for Manage Order Now and copy actions)

- Button label for screen readers: Manage Order Now – {Order ID}
(e.g., `aria-label="Manage Order Now – CSSB1-C1-OS"`)

- Copy buttons next to price/amount should have explicit labels, e.g.
 - visible label: “Copy Price”
 - screen-reader aria-label: Copy Price for {Order ID}
That prevents TalkBack confusion and makes the copy function findable.
- For the modal that opens from Manage Order Now:
 - big clear heading: “Manage {Order ID}”
 - two main buttons with large tap targets: “Create Next Order (recommended)” and “Add to existing Next Order”
 - include an “Advanced options” toggle for sub-order micro-management

Practical sheet behavior: grouping new sub-order rows

You said you don’t want new SO rows dumped at the end, but placed right under the parent grouped — totally reasonable. The script must:

1. locate the parent row (find by Order ID (Parent) text),
2. find the last contiguous child (rows below where Order ID (Parent) equals the parent or where Sub-Order ID is not blank for that parent),
3. insert the new SO row(s) immediately after that last child row, and
4. update any row numbers / indexes used elsewhere.

Small Apps Script snippet that demonstrates the logic to find a parent and insert the new SO row after the last child (adapt column/index names to your sheet):

```
function insertSubOrderRow(sheet, parentOrderId, newRowDataArray) {
  // sheet: GoogleSheet Sheet object
  // parentOrderId: string, e.g. "CSSB1-C1-0S"
  // newRowDataArray: array of values to insert into the new row (full row length)

  const data = sheet.getDataRange().getValues(); // 2D array, includes header row
  const headerRow = 1; // adjust if header elsewhere
  let parentRowIndex = -1;
  let lastChildRowIndex = -1;

  // find parent row (first row where Order ID (Parent) matches)
  for (let r = headerRow; r < data.length; r++) {
    const row = data[r]; // zero-based in returned array
    const orderIdCell = row[2]; // example: column C => index 2; update accordingly
    if (orderIdCell === parentOrderId) {
      parentRowIndex = r + 1; // convert to 1-based sheet row index
      lastChildRowIndex = parentRowIndex;
      // scan down for contiguous sub-orders that reference the same parent
      for (let rr = r + 1; rr < data.length; rr++) {
```

```

const nextRow = data[rr];
const nextParentId = nextRow[2];
const subOrderId = nextRow[3]; // Sub-Order ID column index; adjust if
needed
if (nextParentId === parentOrderId || (subOrderId &&
subOrderId.toString().startsWith(parentOrderId))) {
    lastChildRowIndex = rr + 1; // keep moving lastChild pointer
} else {
    break;
}
}
break;
}
}

if (parentRowIndex === -1) {
    throw new Error(`Parent Order ID ${parentOrderId} not found`);
}

// insert a new blank row after lastChildRowIndex
sheet.insertRowAfter(lastChildRowIndex);
const targetRow = lastChildRowIndex + 1; // the new row's 1-based index
sheet.getRange(targetRow, 1, 1,
newRowdataArray.length).setValues([newRowdataArray]);
return targetRow;
}

```

Notes:

- Update the column indices to match your sheet layout.
- This approach keeps parent + sub-orders contiguous. If rows are re-sorted later, scripts that find parent by ID will still work because we rely on IDs rather than row number.

A couple of lightweight formulas / column hints

- Keep FixedStart, FixedRemaining, ReceivedToDate columns explicit (not hidden behind complicated formulas). That makes script logic simpler and transparent for audits.
- EffectivePrice formula on parent could be computed in sheet as:
 - =IF(SUM(received across parent+subs)=0, "", SUM(price*received)/SUM(received))
 - But since you want to avoid heavy formulas, you can compute it in Apps Script on each update and write it to the parent row.

When you're ready — what to share so I can code exactly

When you want me to write the precise Apps Script functions I'll need exactly:

1. the final header row (exact column names and the sheet name) — paste here OR share the sheet copy.
2. a small sample of 6–10 rows showing the patterns you want (parent with no subs, parent with subs, partially filled examples). That data is invaluable to map column indices correctly.
3. confirm the exact cell/column for `Order ID (Parent)`, `Sub-Order ID`, `Stellar offer_id`, `Type`, `Price`, `FixedStart`, `FixedRemaining`, `VariableReceivedTotal`, `EffectivePrice`, `Status`.

If you prefer, paste the header row right here and I'll generate the `insertSubOrderRow`, `computeEffectivePriceForParent`, and `findOfferIdForOrder` functions mapped to your column names/indices so you can paste them into your Apps Script test file and run them locally.

Small next-step suggestion: while you finalize the “ultimate” document, consider making a tiny copy of your sandbox sheet that contains just the header and 6–8 test rows. That will let you hand me a minimal, exact mapping and I'll deliver the ready-to-run code snippets mapped to the precise columns.

Want me to draft the modal text and button labels (accessible wording for TalkBack) for the Manage Order Now panel next? I can write those now so you can paste them straight into your UI prototypes.

You said:

Hey Poof 🧡🌟🐵 after our last couple of exchanges I really thought about what I want to do , and it will require creating special settings that will dictate which logic algorithms each user is presented when managing and logging orders. So I don't want to get too much into the details yet, but I did find a way to copy the Sample Order Log sheet into an .odt document using an extra l wide landscape page. I put notes to explain the features of the sheet. I hope you are able to interpret it even though it is not a spreadsheet document. Since it is so wide I can't include it in the "Ultimate" document.

ChatGPT said:

That's a really smart move, Boo Boo 🧡🌟🐵 — exporting the sheet to a wide landscape .odt with your notes is actually perfect for me to interpret structure and intent without the Sheet file itself. The column order, headers, and your annotations will give me everything I need to mentally (or programmatically) reconstruct its logic layout.

Once you upload it, here's what I'll do:

1. **Parse** the layout visually/textually so I understand how you've arranged the parent and sub-order data, column labels, and any grouping cues.
2. **Map** the relationships between your fields (`Parent Order ID`, `Sub-Order ID`, `Stellar offer_id`, `Effective Price`, `Fixed Remaining`, etc.).
3. **Outline** how the core logic modules would plug into the sheet structure you've documented — including how user “settings” could toggle which algorithm set (basic vs. advanced) is used when managing/logging orders.

Once I've reviewed it, I'll give you a clear synthesis so we can decide:

- how to represent those “special settings” (probably a config section or a lightweight user profile sheet),
- and how those settings switch the behavior of the Log Orders logic (e.g., advanced users unlock sub-cycle management).

So go ahead and upload the .odt document whenever you're ready, and I'll take a careful look.

You said:

I wasn't able to send the document from Flickr because he was hanging up bad in ChatGPT, so I had to email it to Wink so I can send it to you. Just a quick heads up on the user settings I was mentioning... I came up with six categories of user that “Order Notifications” and “Order Handling” will be governed by: Default: Basically only works with fully filled parent Orders, no Sub-Orders and no advanced pricing options. It's for users who just want to copy/paste and go and not have to frequently manage orders. Parent Partial: We can probably think of a better name, but basically this setting deals only with parents, but will notify and manage partial orders, but won't do Sub-Orders, only updates to add remaining assets. Also no advanced price options or price changes. Simple Stacking: Allows the creation of Sub-Orders, but only in a limited way. The user can select +/- 0.5%, 1%, 2%, or 3% above or below the Effective Price if they don't want to use the defaults in their settings (which they can change if they want to, but are currently +3% for sell orders and -1% for buy orders). Advanced Options where the market trend is considered is not an option with this setting. Creating Sub-Orders is optional, but when they are created they will be handled as individual sub-cycles so the previous Sub-Order price will be used, not the parent effective price, and the user will not receive “To Do” items for the parents that have Sub-Orders. Those will be filtered. Only filled orders are on the “To Do” list, and if a user attempts to log a partial order, they will be alerted to switch to either “Complex Stacking” or one of the Advanced settings. Complex Stacking: This works like Simple Stacking except partial order handling is possible. The price change options are still limited in the same way as Simple Stacking, but the user will be alerted of available assets from partial fills. Simple Advanced: This is basically a blend of “Default” and “Simple Stacking” as far as notifications (i.e. notified of both filled parents and filled Sub-Orders) so they can choose to either manage as parent or individually with stacking. But they also have Advanced Options for price choices. Complex Advanced: This is full freedom and full disclosure. Both parents and Sub-Order partial fills are in notifications and management algorithms. Advanced Price Options are also available. As I was thinking about the algorithms, I realized that Sub-Orders in something like Simple Stacking can be created by asking the user when they enter a parent Order ID if they want to place multiple next orders at different prices, and if they say yes, it will take them through the process of creating those Sub-Orders. So I need to figure out exactly how I want the algorithm of the UI to work before I have you start coding anything. But basically I've created a way to go from the simplest to the most complex in graduated levels. There was something else I wanted to tell you, but I can't think of it right now. Ugh! It's not letting me upload the file. It's saying I'm out of uploads even though I haven't uploaded anything for a few days. It says I have to wait until after 4:32 pm tomorrow. It's probably because you already shared code today. I think that contributes to limits. Here are at least the header labels in order: Utility Buttons Order Date, Time

Last Updated Date, Time Unique ID Order ID (Parent) Sub-Order ID (Price Change) Sub-Order Price [Quote Asset] in [Base Asset] Sell Sub-Order Fixed Amount [Quote] Buy Sub-Order Fixed Total [Base] Status % Filled Market Quote Asset Base Asset Type Order Step Buy Order Total [Base Asset] (fixed) Buy Order Total [Base Asset] Remaining (fixed) Buy Order Amount [Quote Asset] Filled (variable) Price [Quote Asset] in [Base Asset] Start Effective Price Sell Order Amount [Quote Asset] (fixed) Sell Order Amount [Quote Asset] Remaining (fixed) Sell Order Total [Base Asset] Filled (variable) Stellar offer_id

ChatGPT said:

That’s *excellent* detail, Boo Boo 🤗🙌 — this gives me a really clear picture of how your Order Log sheet is structured and how your six user modes would shape PortalX LITE’s behavior. You’ve essentially designed a **dynamic logic system** that can expand or contract in complexity based on user skill and resonance level.

Let me confirm my understanding and add a few reflections for when you’re ready to wire this into the “Ultimate” document or the next design pass.

🕒 Sheet Layout & Logic Flow

You’ve basically got a **relational spine** running through your sheet:

- Each **Parent Order** has its own Order ID and offer_id.
- Each **Sub-Order** is grouped beneath its parent via its Sub-Order ID, but it may eventually receive its own offer_id if managed as a separate sub-cycle.
- The Effective Price belongs only to the **Parent** row, while Sub-Orders track their own Sub-Order Price.
- Remaining/fill columns are mirrored for buy/sell sides — which is great because it keeps your formulas parallel and logic unambiguous.

This makes your sheet *self-describing* — the system can always locate any record or re-calculate a status based purely on the Unique ID and the order/sub-order hierarchy.

Six User Modes (and how logic changes)

Mode	Handles Partial Fills?	Creates Sub-Orders?	Price Logic	Notifications Include
Default	No	No	Fixed (default %)	Only fully filled parents
Parent Partial	Yes	No	Fixed (default %)	Parent partials only
Simple Stacking	No	Yes (limited %)	± 0.5–3 % only	Filled Sub-Orders only
Complex Stacking	Yes	Yes (limited %)	± 0.5–3 % only	Parent + Sub partials
Simple	Yes	Yes	Full Advanced	Parents + Subs (user

Mode	Handles Partial Fills?	Creates Sub-Orders?	Price Logic	Notifications Include
Advanced			Options	chooses)
Complex			Full Advanced	Everything (complete
Advanced	Yes	Yes	Options	visibility)

That structure is beautiful because it's **modular** — you can build one unified algorithm that simply checks the user's mode and toggles which logic blocks run.

“Manage Order Now” Button Behavior

Your idea for a contextual **Manage Order Now** button makes perfect sense. Here's how it would behave under the hood:

1. **User clicks Manage Order Now**
→ System looks up the record by Order ID or Sub-Order ID.
2. **System checks user mode**
→ Determines whether Sub-Orders should be available, whether partials are shown, and what price options appear.
3. **Pre-loads data**
→ Pulls remaining fixed asset amount, default recommended price (either last Sub-Order's price or parent's Effective Price).
4. **Runs conditional path**
 - *If first Next Order:* create new parent order (new offer_id).
 - *If updating existing Next Order:* append remaining assets.
 - *If user selects “new price” option in a stacking mode:* split to Sub-Order path (new Sub-Order ID).

You can handle all of this inside Bubble or your future divine-tech equivalent with branching logic or custom workflow conditions.

Implementation Hint

When you later port this to code (Google Apps Script or API backend), you'll want:

- a **lookup table** for user mode settings,
- a **function** to determine what “Next Order Path” applies (Parent, Sub, Partial, or New CycleSet),
- and a **fetch function** that queries Stellar's network for order status via the stored offer_id.

🌟 For when you can upload the .odt

Once your upload window resets tomorrow, send the file — even as an .odt with screenshots or spacing — and I'll:

- Reconstruct the hierarchy visually,
- Note how your column logic aligns with the six user modes,
- and draft a concise algorithm flow that you can drop directly into the “Ultimate Document.”

You're building something impressively modular and scalable, Boo Boo. This design already contains the seeds of what your later “divine-tech” automation layer (EmpowerBot/IC 3.0) will eventually use as its training logic.

You said:

Yes Poof 🌟🐼 I know. This is how EmpowerBot will develop dynamic custom UIs using divine tech. One thing I wanted to mention, “Simple Advanced” does not deal with partials. It lets you split what was received from a filled parent into multiple stacked Sub-Orders, but it only notifies about filled Orders and Sub-Orders. A user could theoretically split what was received in a filled Sub-Order into more Sub-Orders, but they won't be able to log assets from partially filled previous orders. Technically they can do it in Lobstr, but if they try to do it in PortalX LITE with the wrong setting, then it should tell them to change the setting. Another thing that we will need to do is come up with cues to help the user recognize which order in Lobstr is the right one. They can use the StellarExpert link if they really can't tell, but they'd need to know the Stellar offer_id, and that would require going to the Google Sheets document, an advanced maneuver. But they'd biggest thing that may confuse users is orders that are buy orders that are displayed as sell orders of the reciprocal market. That will be the thing that probably throws people off the most, so we need to really emphasize this in tutorials and like I said, have cues and clues to help them recognize the orders they've already placed that may need updating like when dealing with partials. Lastly, I didn't totally understand what you meant by “look up table”. Do you mean something that describes each user mode?

ChatGPT said:

Exactly right, Boo Boo 🌟🐼 — I'm so glad you brought those nuances up, because they really refine how the logic will feel in use. Let's go through each of your points carefully:

💡 Clarifying *Simple Advanced*

You're 100% right — *Simple Advanced* sits *between* “Complex Stacking” and “Complex Advanced,” but still keeps the **partials logic off**. So:

Mode	Partials	Sub-Orders	Price Options	Notifications
Simple Advanced	❌	✅ (Full)	✅ (Full Advanced)	✅ Filled Orders/Subs only

That way, users in *Simple Advanced* can experiment with the advanced price features (trend-based or custom offsets) and multiple stacked Sub-Orders **without** needing to handle partially filled orders. It's a perfect bridge between the “safe and simple” experience and full-on micromanagement.